



Analyzing Cyber Attack Vectors With Honeypots

Madalena Bordadágua
Madalena Martins
Mariana Antunes

Project Report for the 1st Cycle Integrative Project in
**Bachelor in Telecommunications and Informatics
Engineering**

Supervisor: Prof. Fernando Mira Silva

February 2026

Resumo

A recolha de informação sobre ciberameaças depende cada vez mais de tecnologias de decepção para capturar e analisar o comportamento dos atacantes. No entanto, a implementação destas infraestruturas em redes académicas ou empresariais sujeitas a rigorosos controlos de segurança apresenta desafios arquitetónicos significativos. Este projeto apresenta o desenho, a implementação e a avaliação de uma infraestrutura híbrida de múltiplos *honeypots* integrando o Cowrie, o Dionaea e serviços de reencaminhamento leves (Socat), especificamente adaptada para operar num ambiente institucional restrito.

Para apoiar a recolha e análise de dados, a arquitetura incorpora uma *pipeline* de telemetria centralizada baseada na *Stack* ELK (Elasticsearch, Logstash e Kibana), complementada pela classificação automatizada de *malware* através da API do VirusTotal. Devido às políticas da *firewall* institucional, que restringem a exposição de vários protocolos tipicamente utilizados em ambientes empresariais, a superfície de ataque externa foi adaptada para priorizar serviços SSH, HTTP/HTTPS e bases de dados selecionadas. Simultaneamente, a infraestrutura foi posicionada de modo a fornecer visibilidade sobre potenciais atividades de reconhecimento interno e movimento lateral dentro da rede do campus.

Durante um período de monitorização de 84 dias, o sistema capturou e processou mais de 583.000 eventos de segurança com origem em 2.966 endereços IP únicos. A telemetria recolhida revelou uma predominância de atividade automatizada de *botnets* focada em tentativas de autenticação por força bruta, reconhecimento de sistemas e preparação para persistência, em vez de exploração avançada de *software* ou implementação de *malware*. Embora não tenham sido capturadas cargas maliciosas com sucesso, a infraestrutura forneceu perspectivas valiosas sobre o comportamento das campanhas contemporâneas de ataques automatizados.

Em suma, o projeto valida uma arquitetura de decepção escalável, capaz de apoiar a recolha de inteligência de ameaças, traçar o perfil da atividade global de ataques e melhorar a visibilidade sobre serviços de rede expostos a operar dentro de ambientes institucionais restritos.

Palavras-chave: Cibersegurança, Honeypots, Inteligência de Ameaças, Stack ELK, Botnets, Segurança de Redes.

Abstract

Cyber threat intelligence increasingly relies on deception technologies to capture and analyze attacker behavior. However, deploying such infrastructures within academic or enterprise networks subject to strict security controls presents significant architectural challenges. This project presents the design, deployment, and evaluation of a hybrid multi-honeypot infrastructure integrating Cowrie, Dionaea, and lightweight relay services (Socat), specifically adapted to operate within a restricted institutional environment.

To support data collection and analysis, the architecture incorporates a centralized telemetry pipeline

based on the ELK Stack (Elasticsearch, Logstash, and Kibana), complemented by automated malware classification through the VirusTotal API. Due to institutional firewall policies restricting the exposure of several enterprise-oriented protocols, the external attack surface was adapted to prioritize SSH, HTTP/HTTPS, and selected database services. Simultaneously, the infrastructure was positioned to provide visibility into potential internal reconnaissance and lateral movement activity within the campus network.

During an 84-day monitoring period, the system captured and processed more than 583,000 security events originating from 2,966 unique source IP addresses. The collected telemetry revealed a predominance of automated botnet activity focused on brute-force authentication attempts, system reconnaissance, and persistence preparation rather than advanced software exploitation or malware deployment. Although no malware payloads were successfully captured, the infrastructure provided valuable insights into the behavior of contemporary automated attack campaigns.

Overall, the project validates a scalable deception architecture capable of supporting threat intelligence collection, profiling global attack activity, and improving visibility into exposed network services operating within constrained institutional environments.

Keywords: Cybersecurity, Honeypots, Threat Intelligence, ELK Stack, Botnets, Network Security.

Contents

Resumo	ii
Abstract	ii
List of Tables	vii
List of Figures	vii
Glossary	viii
1 Introduction	1
1.1 Motivation	1
1.2 Topic Overview	2
1.3 Objectives and Contributions	2
1.4 Thesis Outline	3
2 State of the Art	5
2.1 Background	5
2.1.1 Honeypots: Concept and Role	5
2.1.2 Types of Honeypots	6
2.1.3 Honeypots Landscape: Commercial vs. Open Source	6
2.1.4 Advanced Cybersecurity Concepts	7
2.2 Technologies	7
2.2.1 Comparison of Evaluated Honeypots	7
2.2.2 The SSH and Telnet Protocols	8
2.2.3 The Server Message Block (SMB) Protocol	9
2.2.4 Web Protocols: HTTP, HTTPS, and HTTP-Alt	9
2.2.5 Database Protocols: MySQL, PostgreSQL, and MSSQL	9
2.2.6 Remote Desktop Protocol (RDP)	10
2.3 Related Work	10
3 Architecture	13
3.1 System Overview and Evolution	13
3.2 Multi-Honeypot Architecture	14
3.3 Attacker Flow: Finite State Machine (FSM)	15
3.4 Data Analysis Stack (ELK Pipeline)	17

3.5	Institutional Deployment and Network Considerations	17
3.6	Threat Intelligence Integration	19
4	Implementation	21
4.1	Deployment Environment and Initial Configuration	21
4.2	SSH Traffic Redirection and Service Exposure	21
4.3	Cowrie Customization and HoneyFS Enhancements	22
4.4	Dionaea Deployment and Multi-Service Expansion	23
4.5	Malware Capture	23
4.6	Logging, Monitoring, and Data Visualization	23
4.7	Challenges and Deployment Limitations	24
4.8	Security and Containment Measures	24
5	Final Tests and Results	27
5.1	Experimental Methodology and Global Traffic Overview	27
5.2	Attack Surface and Targeted Services	29
5.2.1	Web and Database Probes (Dionaea)	29
5.2.2	Port Scanning and Low-Interaction Decoys (Socat)	31
5.2.3	Terminal Services (Cowrie)	31
5.3	Threat Profiling and Geolocation	33
5.3.1	Geographical Distribution	33
5.3.2	Threat Actor Decentralization	34
5.4	Credential Exploitation (Brute-Force Analysis)	35
5.5	Post-Authentication Behavior and Command Analysis	37
6	Discussion and Conclusions	41
6.1	Discussion and Conclusions	41
6.2	Future Work	42
6.2.1	Transition to High-Interaction Honeypots	42
6.2.2	Dynamic Cryptographic Deception	43
6.2.3	Advanced Malware Analysis and Sandboxing	43
6.2.4	Deployment Outside Institutional Network Constraints	43
	Bibliography	45

List of Tables

3.1	Diagnostic of external port accessibility and IST firewall behavior.	18
5.1	Global Experimental Dataset Metrics	28
5.2	Network-Layer Traffic Metrics for Socat Decoys	31
5.3	Top Observed Authentication Attempts	36
5.4	Top Executed Commands Captured within the Interactive Shell	37

List of Figures

3.1	Global Architecture and Data Flow Diagram	14
3.2	Finite State Machine Diagram of Attacker Interaction	16
5.1	Temporal Distribution of Attack Events	29
5.2	Distribution of Attacked Network Services (Dionaea)	30
5.3	Distribution of Attacked Network Services (Cowrie)	32
5.4	Geographical Origin of Attacks	34
5.5	Top Source IP Addresses	35
5.6	Distribution of Most Attempted Brute-Force Credentials	36

Chapter 1

Introduction

In recent decades, the increasing digitalization of systems and infrastructures has significantly expanded the attack surface of organizations. By introducing countless new entry points through cloud services and remote protocols, this digital shift has driven cyberattacks to become more frequent and sophisticated, posing serious financial, operational, and reputational risks.

Traditional cybersecurity approaches rely mainly on passive defense mechanisms, such as firewalls, Intrusion Detection System (IDS), and antivirus software. These solutions aim to block known threats; however, they present important limitations in detecting unknown attacks. Furthermore, this purely reactive approach leaves defenders blind to the adversaries' actual tools, automated scripts, and post-breach behaviors.

1.1 Motivation

This scenario raises a key challenge: how can attackers be studied without exposing real systems to risk?

To address this problem, this project explores the use of honeypots as a proactive cybersecurity technique. The main objective is to analyze real-world attack vectors by deploying and studying interactions with malicious actors in a controlled environment.

The project is initially focused on the use of Cowrie, a widely adopted Secure Shell (SSH) and Telnet honeypot. This choice is justified by its stability, ease of deployment, and efficacy in capturing a high volume of real-world automated attacks, particularly brute-force attempts. By leveraging this system, it becomes possible to collect meaningful data, extract statistics, and safely analyze attacker behavior.

As the project progressed, it became clear that relying on a single honeypot was insufficient to cover all relevant attack surfaces. For this reason, the architecture was extended to include additional honeypots targeting other protocols and attack vectors, enabling a more comprehensive analysis of malicious activity. Ultimately, this demonstrates that acquiring actionable threat data is not exclusive to large enterprise environments, but an objective that can also be effectively achieved through independent academic research to improve security.

1.2 Topic Overview

To address the limitations of passive defenses, this project evaluates the deployment of honeypots. A honeypot is essentially a decoy system designed to look like a real, vulnerable target. By monitoring interactions with these decoys, it becomes possible to safely observe adversary methods, tools, and objectives without risking real infrastructure.

This study implements a multi-honeypot approach to capture different types of threats. It relies heavily on Cowrie, a system that emulates SSH and Telnet services, which proves highly effective in capturing automated login attempts and command-line exploitation. However, to achieve a broader view of the threat landscape, Dionaea and Socat were integrated into the architecture. These components simulate and expose additional network and database services (such as Hypertext Transfer Protocol (HTTP), MySQL, and PostgreSQL), which are frequently targeted by automated scanners and used to spread malware.

Finally, capturing raw telemetry alone is insufficient; the data must be systematically analyzed. For this reason, all the collected information is processed and visualized using the Elasticsearch, Logstash, and Kibana (ELK) stack (Elasticsearch, Logstash, and Kibana). This pipeline facilitates the transformation of raw, complex logs into clear, actionable intelligence, enabling the geographical mapping of attack origins and the identification of deployed malicious payloads.

1.3 Objectives and Contributions

The main objective of this research is to design, deploy, and analyze a secure multi-honeypot environment to study real-world cyber threats. To achieve this, the following specific analytical and architectural objectives were established:

- **Deploy a multi-protocol environment:** Implement and configure Cowrie, Dionaea, and Socat to capture a diverse range of attack vectors, spanning from interactive terminal access to network-layer exploitation.
- **Improve system realism:** Customize the simulated environments, incorporating realistic hostnames and structured decoy files, to maximize attacker engagement and mitigate sandbox fingerprinting techniques.
- **Set up a data pipeline:** Integrate the ELK stack to parse, enrich, and visualize the generated logs, facilitating the identification of attack origins, credential exploitation trends, and automated scanning patterns.
- **Automate malware analysis:** Interface the infrastructure with the VirusTotal Application Programming Interface (API), which acts as a centralized intelligence platform that aggregates detection data from dozens of industry-leading antivirus engines and website scanners. This integration enables the system to automatically cross-reference the cryptographic hashes of captured files,

allowing for the real-time classification, identification of known malware families, and operational profiling of payloads introduced by threat actors during post-authentication phases.

The expected deliverables for this project include a fully working and continuously monitored honeypot infrastructure, as well as visual dashboards that map and summarize the captured attack data in an accessible way. Ultimately, these practical efforts culminate in this dissertation, which details the system setup, the challenges faced during implementation, and the conclusions drawn from the collected data.

1.4 Thesis Outline

The remainder of this document is structured as follows. Chapter 2 establishes the theoretical foundation of the project by defining the concept and classifications of honeypots, detailing the targeted network protocols, and exploring advanced cybersecurity concepts such as cryptographic authentication mechanisms, sandbox evasion, and honeypot fingerprinting techniques. Chapter 3 presents the architectural design of the proposed infrastructure, describing the multi-honeypot deployment model, the attacker interaction workflow represented through a Finite State Machine (FSM), the centralized telemetry processing pipeline, and the threat intelligence integration mechanisms. Chapter 4 details the practical implementation of the infrastructure, including network traffic redirection strategies, Docker-based isolation mechanisms, HoneyFS customization procedures, deployment adaptations required by institutional perimeter restrictions, and the integration of monitoring and malware-analysis components. Chapter 5 evaluates the empirical results obtained during the monitoring period, analyzing attack distributions, targeted services, credential exploitation attempts, geographical threat patterns, and post-authentication attacker behavior observed across the deployed honeypots. Finally, Chapter 6 synthesizes the principal findings of the project, discusses the limitations of the implemented architecture, evaluates the effectiveness of the deployed deception mechanisms, and proposes directions for future research and infrastructure expansion.

Chapter 2

State of the Art

This chapter provides the theoretical foundation required to understand the concepts, technologies, and methodologies applied throughout this project. It begins by introducing honeypots as active cyber defense mechanisms and discussing their classifications, operational objectives, and role in cybersecurity. Section 2.1 presents the background concepts necessary to understand deception technologies, including honeypot types, deployment approaches, and advanced cybersecurity concepts related to realism and attacker detection. Section 2.2 examines the technologies and network protocols targeted by the deployed infrastructure, including SSH, Telnet, SMB, web services, database services, and remote administration protocols, as well as the honeypot solutions selected for their monitoring. Finally, Section 2.3 reviews relevant academic literature on honeypot deployments, cyber deception strategies, telemetry collection, and threat intelligence integration, providing the research context that motivated the design choices adopted in this project.

2.1 Background

2.1.1 Honeypots: Concept and Role

Honeypots are an important technique within the field of active cyber defense. These systems consist of intentionally exposed computational resources designed to appear vulnerable in order to attract malicious actors and study their behavior [1].

A honeypot can be defined as a system whose primary purpose is to be probed, attacked, or compromised, enabling the collection of valuable information regarding malicious activities, exploitation techniques, and intrusion patterns.

Unlike traditional defensive technologies, honeypots are not primarily intended to block attacks. Instead, they focus on monitoring and analyzing adversary interactions. This approach enables researchers and security practitioners to observe real-world threats in controlled conditions, including automated scanning campaigns, brute-force attacks, malware delivery attempts, and post-compromise activity.

By operating as controlled deception environments, honeypots provide visibility into attacker methodologies that would otherwise remain difficult to observe through conventional passive security mechanisms alone.

2.1.2 Types of Honeypots

Honeypots can be classified according to both their interaction level and operational purpose. Based on interaction level, honeypots are generally divided into three categories [2]:

- **Low-interaction honeypots** simulate only limited services and restrict attacker activity. Although they provide reduced behavioral visibility, they are relatively safe and simple to maintain.
- **Medium-interaction honeypots**, such as Cowrie, emulate services with a greater degree of realism, allowing attackers to interact with simulated environments while maintaining controlled isolation. These systems provide significantly richer telemetry without exposing the risks associated with fully compromised operating systems.
- **High-interaction honeypots** consist of real systems that allow unrestricted attacker interaction. They provide highly detailed behavioral data at the cost of increased operational complexity and security risk.

From a functional perspective, honeypots may also be classified as:

- **Research honeypots**, primarily used in academic and experimental environments to analyze attack techniques, adversary behavior, and emerging threats.
- **Production honeypots**, deployed within operational infrastructures to complement existing security controls by detecting and monitoring malicious activity in real time.

Each category presents different trade-offs between realism, operational safety, maintenance complexity, and the quality of collected threat intelligence.

2.1.3 Honeypots Landscape: Commercial vs. Open Source

Organizations and researchers deploying deception technologies must typically choose between commercial and open-source solutions, each offering distinct advantages and limitations.

Commercial honeypots are enterprise-oriented deception platforms, such as Fortinet FortiDeceptor [3], SentinelOne Singularity Identity [4], and TrapX [5]. These solutions are designed for large-scale operational environments and usually provide centralized management, automated threat intelligence integration, dedicated technical support, and advanced analytics interfaces. However, their proprietary nature limits deep customization and internal visibility, while licensing costs may restrict their accessibility for academic research.

In contrast, open-source honeypots such as Cowrie [6], Dionaea [7], and Honeyd [8] offer complete transparency and high configurability. Researchers can inspect source code, customize simulated

filesystems, modify service behavior, and develop tailored integrations according to experimental requirements. Although these platforms demand greater technical expertise for deployment and maintenance, their flexibility makes them particularly suitable for research-oriented environments.

For this reason, open-source honeypots remain the preferred choice for experimental cybersecurity studies focused on behavioral analysis and infrastructure customization.

2.1.4 Advanced Cybersecurity Concepts

To effectively deceive attackers and collect meaningful telemetry, honeypot environments must incorporate advanced cybersecurity concepts while minimizing indicators that reveal their artificial nature.

Cryptography and Certificates

Protocols such as SSH rely on asymmetric cryptography [9], which operates through pairs of mathematically related keys: a public key, which can be openly distributed, and a private key, which remains secret. Similarly, encrypted web communications depend on TLS/SSL certificates, digital documents that bind a public cryptographic key to a server's identity to enable secure and authenticated connections [9].

When simulating services such as SSH or HTTPS, honeypots must present realistic cryptographic credentials. The use of default keys, invalid certificates, or widely recognized signatures may expose the environment as a deception system. Consequently, properly customized cryptographic artifacts are essential to maintaining the credibility of the simulated infrastructure.

Sandbox Evasion and Fingerprinting

Modern cyberattacks are frequently automated and often include mechanisms designed to detect whether they are interacting with real systems or controlled analysis environments, such as honeypots. This process is commonly referred to as fingerprinting [10].

Attackers may inspect hostnames, filesystem structures, command outputs, process behavior, or network responses to identify inconsistencies associated with honeypots. For example, default hostnames or unrealistic command execution behavior can quickly reveal the presence of a simulated environment.

Since many honeypots rely on sandboxed or partially emulated systems, certain commands may produce incomplete or inconsistent outputs. When such anomalies are detected, automated malware frequently terminates execution to avoid analysis.

As a result, reducing fingerprinting indicators through environmental customization is a critical requirement for improving attacker engagement and increasing the quality of collected behavioral data.

2.2 Technologies

2.2.1 Comparison of Evaluated Honeypots

Different honeypot architectures are designed to capture distinct categories of malicious activity depending on the protocols and services they emulate. This project evaluates multiple complementary honeypot technologies to increase attack surface coverage and improve visibility into different intrusion stages.

Cowrie is a medium-interaction honeypot that emulates SSH and Telnet services. It is particularly effective at capturing brute-force authentication attempts, command execution behavior, and malware download activity [11]. Due to its balance between realism and operational safety, it is widely adopted for monitoring attacks targeting remote administration services.

Dionaea is a low-to-medium interaction honeypot focused on malware collection and network-oriented exploitation attempts [12]. It supports multiple protocols, including SMB, FTP, HTTP, and MySQL, enabling the observation of malware propagation techniques and services exploitation behavior, particularly within Windows-oriented environments.

Honeyd operates as a framework capable of simulating multiple virtual hosts and network services simultaneously. It is especially useful for capturing reconnaissance activity, port scanning behavior, and network fingerprinting attempts performed during the early stages of intrusion campaigns [8].

Socat is a lightweight network relay utility that, within this architecture, operates as a low-interaction decoy layer. It was used to expose supplementary services such as Remote Desktop Protocol (RDP), Microsoft SQL Server (MSSQL), PostgreSQL, and HTTP-Alt without deploying full application stacks. This approach enables the efficient capture of indiscriminate scanning and enumeration attempts while maintaining low resource consumption [13].

Each technology contributes different analytical capabilities. Cowrie provides detailed interaction telemetry but is limited to terminal-oriented attacks. Dionaea expands visibility toward malware delivery and network exploitation attempts. Socat broadens the observable attack surface through lightweight service exposure.

Combining multiple honeypot technologies therefore enables a more comprehensive analysis of attack vectors across different protocols and phases of the intrusion lifecycle.

2.2.2 The SSH and Telnet Protocols

Understanding the protocols emulated by the deployed infrastructure is essential for analyzing the captured attack vectors. Both Secure Shell (SSH) and Telnet are widely used for remote system administration, making them frequent targets for unauthorized access attempts.

Telnet, operating on port 23, is a legacy protocol that transmits data in plain text without encryption. Although considered insecure by modern standards, it remains common in legacy systems and Internet of Things (IoT) devices. Due to weak default credentials and outdated firmware, Telnet-enabled devices are frequently targeted by automated botnets such as Mirai [14], which perform large-scale brute-force attacks to compromise vulnerable systems and recruit them into distributed attack infrastructures.

SSH, operating on port 22, was designed as a secure alternative to Telnet. It uses strong encryption mechanisms to protect communication channels and authentication exchanges. However, despite its cryptographic protections, exposed SSH services remain heavily targeted by automated brute-force and dictionary attacks attempting to exploit weak credentials, particularly when access control is not based on strict public/private key authentication. Because SSH grants direct command-line access to remote systems, compromised accounts may provide attackers with privileged access capable of supporting

lateral movement, malware deployment, and persistence operations.

2.2.3 The Server Message Block (SMB) Protocol

While SSH and Telnet are primarily associated with remote administration, enterprise environments heavily depend on additional protocols for internal operations. One of the most relevant is the Server Message Block (SMB) protocol.

SMB enables resource sharing within a network, particularly in Windows-based environments, allowing systems to access files, directories, printers, and remote services across multiple hosts. Because SMB often provides direct access to sensitive organizational resources, it represents a highly valuable target for attackers. Vulnerabilities or misconfigurations within SMB services may enable unauthorized access, malware propagation, credential theft, and lateral movement across internal networks [15].

A well-known example is the WannaCry ransomware campaign, which exploited SMB vulnerabilities to spread rapidly between systems on a global scale. Deploying honeypots capable of monitoring SMB traffic therefore provides valuable visibility into automated exploitation attempts and malware distribution strategies targeting enterprise infrastructures [12].

2.2.4 Web Protocols: HTTP, HTTPS, and HTTP-Alt

Web protocols represent one of the largest and most actively targeted attack surfaces on the internet. Hypertext Transfer Protocol (HTTP), operating on port 80, and Hypertext Transfer Protocol Secure (HTTPS), operating on port 443, form the foundation of modern web communication.

Due to their ubiquity, these services are rarely blocked by perimeter firewalls, making them attractive channels for reconnaissance, vulnerability exploitation, malware delivery, and command-and-control communication.

Attackers continuously scan web-facing infrastructures searching for vulnerable applications, outdated content management systems, exposed administrative interfaces, and configuration weaknesses [16].

Additionally, HTTP-Alternate (HTTP-Alt), commonly operating on port 8080, is frequently associated with development environments, reverse proxies, application servers such as Apache Tomcat, and administrative web interfaces. These services are often misconfigured or deployed with default credentials, increasing their attractiveness to automated vulnerability scanners.

Consequently, monitoring web-oriented services provides valuable insights into large-scale reconnaissance behavior and automated exploitation campaigns [17].

2.2.5 Database Protocols: MySQL, PostgreSQL, and MSSQL

Databases constitute critical repositories of organizational information, making them high-value targets for cybercriminal activity. The infrastructure deployed in this project monitored three major database technologies: MySQL (port 3306), PostgreSQL (port 5432), and Microsoft SQL Server (MSSQL) (port 1433).

Exposed database services are frequently targeted by automated brute-force tools attempting to exploit default or weak administrative credentials. Successful compromise may enable attackers to exfiltrate sensitive information, manipulate stored data, deploy ransomware, or establish persistence mechanisms.

In addition to credential-based attacks, database services are commonly scanned for known vulnerabilities capable of enabling remote code execution or privilege escalation.

For these reasons, monitoring database-oriented traffic provides important visibility into automated reconnaissance activity and opportunistic exploitation behavior.

2.2.6 Remote Desktop Protocol (RDP)

The Remote Desktop Protocol (RDP), operating on port 3389, enables remote graphical access to systems, primarily within Windows environments.

Because RDP frequently grants administrative-level access to hosts, it represents a critical target for ransomware operators and intrusion campaigns seeking initial access into organizational infrastructures.

Attackers routinely scan the internet searching for exposed RDP endpoints, which may subsequently be targeted through brute-force attacks, credential stuffing techniques, or protocol-level vulnerabilities.

Monitoring RDP-oriented traffic is therefore valuable for identifying reconnaissance behavior, unauthorized authentication attempts, and the early stages of lateral movement activity [12].

2.3 Related Work

Several academic studies have explored the use of honeypots for cyber threat intelligence collection, malware monitoring, and attacker behavior analysis.

Regarding attacker engagement, the paper *Towards Bio-Inspired Cyber-Deception: A Case Study of SSH and Telnet Honeypots* [18] explored deception strategies aimed at increasing attacker interaction time. The study showed that modifications to filesystem structures, terminal behavior, and system responses reduced honeypot detection rates and encouraged additional post-authentication activity. These findings strongly influenced the realism enhancement strategies implemented throughout this project.

Expanding on this need for realism, another important study is *Q-Cowrie: An Adaptive Honeypot to Analyse Attackers' Behaviour* [19]. This work proposed an adaptive version of Cowrie capable of dynamically modifying its responses according to attacker interaction patterns. The authors demonstrated that increasing environmental realism improves attacker engagement and enhances the quality of collected telemetry. Although the current project shares similar objectives regarding deception realism, it differs by focusing on static HoneyFS customization through realistic artifacts such as SSH keys, configuration files, and enterprise-style hostnames.

An important broad research direction involves expanding observable attack surfaces without introducing excessive infrastructure complexity. The survey *A Survey of Honeypots and Honeynets for*

Internet of Things, Industrial Internet of Things, and Cyber-Physical Systems [13] argued that deploying dedicated virtualized services for every protocol may become computationally expensive and operationally difficult to maintain. Lightweight relay mechanisms, such as Socat, offer an efficient alternative for exposing additional services while preserving resource efficiency and isolation boundaries.

Research involving specific malware-oriented honeypots is also highly relevant. The study *Dionaea Honeypot Implementation and Malware Analysis in Cloud Environment* [12] investigated the deployment of Dionaea within a cloud infrastructure to monitor malware propagation through vulnerable network services such as SMB. The authors demonstrated how Dionaea can capture malicious binaries, isolate payloads, and support automated malware intelligence collection. Unlike that deployment, which operated in an unrestricted cloud environment, the present project was constrained by institutional firewall policies that limited SMB exposure, shifting the monitoring focus toward HTTP and HTTPS traffic.

Additional research has highlighted the importance of centralized telemetry processing within honeypot ecosystems. Multiple studies [20, 21] demonstrate that integrating deception infrastructures with platforms such as the ELK Stack (Elasticsearch, Logstash, and Kibana) significantly improves event correlation, telemetry visualization, and large-scale threat analysis capabilities.

Furthermore, the integration of external Threat Intelligence (TI) platforms has become increasingly relevant in modern honeypot deployments. The study *Intercepting and Monitoring Potentially Malicious Payloads with Web Honeypots* [17] emphasized that standalone honeypots often struggle to classify unknown or polymorphic malware efficiently. By integrating automated TI feeds such as VirusTotal, captured payloads can be rapidly classified through cryptographic hash analysis. This project adopts a similar approach to automate malware identification and reduce dependence on manual reverse engineering procedures.

Finally, looking at recent structural implementations, a relevant contribution is the dissertation developed by Diogo Braz [16], which focused on scalable honeypot infrastructures for automated threat collection. The work explored isolated deception environments combined with centralized monitoring and secure service separation. It also emphasized the importance of modular architectures capable of supporting large-scale event collection over extended periods. While Braz concentrated primarily on scalability and orchestration, the present project focuses more heavily on environment realism and behavioral customization within a restricted institutional network.

Despite the considerable advances described throughout the literature, limited research has explored the deployment of hybrid honeypot infrastructures under highly restricted institutional network conditions. Furthermore, relatively few studies combine deception realism customization, lightweight service exposure, centralized telemetry processing, and automated threat intelligence enrichment within a single integrated architecture.

This project addresses these gaps through the deployment and evaluation of a modular multi-honeypot infrastructure specifically adapted to a constrained academic environment while maintaining continuous monitoring, behavioral analysis, and automated threat intelligence integration.

Chapter 3

Architecture

This chapter describes the structural design and logical organization of the implemented system. The architecture was developed using a modular approach, allowing the infrastructure to evolve from a single honeypot deployment into a multi-service environment capable of monitoring different categories of malicious activity. Section 3.1 explains the system's evolution and protocol selection. Section 3.2 details the parallel multi-honeypot architecture. Section 3.3 models the attacker interaction flow using a Finite State Machine (FSM). Section 3.4 presents the data analysis stack and telemetry pipeline. Section 3.5 discusses the deployment constraints imposed by the institutional network, and finally, Section 3.6 describes the automated threat intelligence integration.

3.1 System Overview and Evolution

The architecture of this project evolved incrementally throughout the development process, following the objective of increasing the visibility of different attack vectors. Initially, the infrastructure was centered exclusively around the Cowrie honeypot, deployed to monitor SSH-based services and terminal-oriented attack activity. This first phase provided a stable foundation for collecting telemetry related to brute-force authentication attempts, automated bot behavior, malicious command execution, and malware download attempts.

Although Cowrie proved highly effective for terminal-based attacks, external access to additional protocols was restricted by the institutional firewall, particularly for Telnet traffic. As a consequence, attacks targeting other network services could not be properly captured or analyzed. To overcome this limitation and broaden the observable attack surface, the infrastructure was extended through the integration of the Dionaea honeypot.

The original objective of this expansion was to monitor SMB-related attacks due to the protocol's importance in enterprise environments and its relevance in ransomware propagation and lateral movement scenarios. However, during deployment, it was verified that inbound SMB traffic was filtered by the institutional firewall, preventing external exploitation attempts from reaching the honeypot environment.

Rather than limiting the project, this restriction led to an architectural adaptation focused on alter-

native externally accessible services, particularly HTTP, HTTPS, and MySQL. This adjustment allowed the infrastructure to continue capturing network-oriented attacks and malware delivery attempts, ultimately resulting in a broader and more effective monitoring environment than the original Cowrie-only deployment.

The final infrastructure therefore combined multiple honeypot technologies with complementary objectives. Cowrie remained responsible for interactive SSH session emulation and terminal-based attack monitoring, while Dionaea extended visibility toward vulnerable network services and automated exploitation attempts. Together, these components provided a more comprehensive perspective of attacker behavior across different protocols and stages of the attack lifecycle.

3.2 Multi-Honeypot Architecture

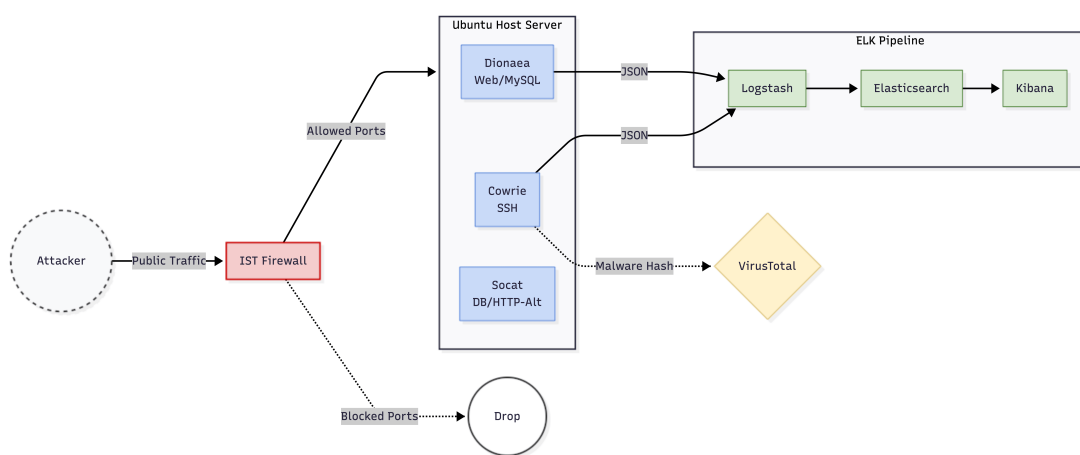


Figure 3.1: Global Architecture and Data Flow Diagram

To avoid service conflicts and ensure efficient traffic capture, the system operates in parallel, dividing the attack surface into three distinct logical blocks:

- Terminal Services Block (Cowrie):** This block handles incoming connections related to remote access services and terminal-based interactions. Natively, Cowrie is designed to emulate both SSH (port 22) and Telnet (port 23) environments. During the deployment, port 22 absorbed the absolute majority of brute-force authentication attempts and automated bot activity. In contrast, external Telnet traffic was restricted by the institutional firewall, positioning port 23 primarily to capture internal probing. Additionally, significant unsolicited traffic targeting port 53 (DNS) was observed. Automated threat actors continuously scan the internet searching for misconfigured "open DNS resolvers" to exploit in Distributed Denial-of-Service (DDoS) amplification attacks. Although Cowrie does not operate as a functional DNS server, logging these inbound probes provided valuable visibility into the scale and origins of this specific reconnaissance behavior. Cowrie therefore acts as the primary interactive component of the system, capturing authentication attempts, executed commands, malware download operations, and post-authentication attacker behavior.

- **Network Services Block (Dionaea):** Operating alongside Cowrie, this component focuses on network-service exploitation attempts and malware propagation activity. While the original deployment objective centered on SMB monitoring, institutional firewall restrictions prevented external SMB traffic from reaching the honeypot environment. As a result, Dionaea was reconfigured to focus primarily on HTTP-based traffic (ports 80 and 443) and MySQL (port 3306), which remained externally accessible during the experimentation phase. This adaptation allowed the infrastructure to successfully capture web-oriented exploitation attempts and malicious payload delivery mechanisms beyond terminal-based attacks.
- **Lightweight Relay Block (Socat):** To further expand the observable attack surface without the resource overhead associated with full application-layer emulation, Socat was introduced as a supplementary low-interaction layer. It exposed additional services such as PostgreSQL (port 5432) and HTTP-Alt (port 8080), successfully capturing indiscriminate network scanning and service enumeration attempts while maintaining minimal resource consumption.

This parallel structure ensures that a broad spectrum of automated attacks can be captured simultaneously without service interference, effectively creating a diversified and scalable decoy network.

3.3 Attacker Flow: Finite State Machine (FSM)

The interaction within a medium-interaction honeypot such as Cowrie is highly dynamic and adapts continuously to the attacker's actions. To model this behavior more clearly, the attacker lifecycle can be represented as a Finite State Machine (FSM) composed of five sequential phases:

- **Connection and Handshake State:** The interaction begins when an inbound TCP connection targets port 22. The honeypot completes the TCP handshake and performs the SSH negotiation process using customized cryptographic keys to reduce basic fingerprinting attempts commonly used by automated malware.
- **Authentication State:** Once credentials are submitted, the honeypot evaluates the authentication attempt. Instead of relying on a traditional credential validation mechanism, the system adopts a permissive authentication policy, accepting a wide range of passwords, particularly for commonly targeted accounts such as `root`, in order to encourage deeper attacker interaction.
- **Interactive Shell (HoneyFS) State:** After authentication, the attacker is placed inside a simulated Linux filesystem known as HoneyFS. At this stage, the environment mimics a legitimate terminal session while safely isolating the real host system. All attacker activity is recorded, including executed commands, filesystem navigation, and interactions with strategically placed decoy files designed to increase realism.
- **Reconnaissance and Execution State:** As attackers execute commands such as `uname`, `whoami`, or `cat /etc/passwd`, the honeypot dynamically generates plausible responses to maintain the

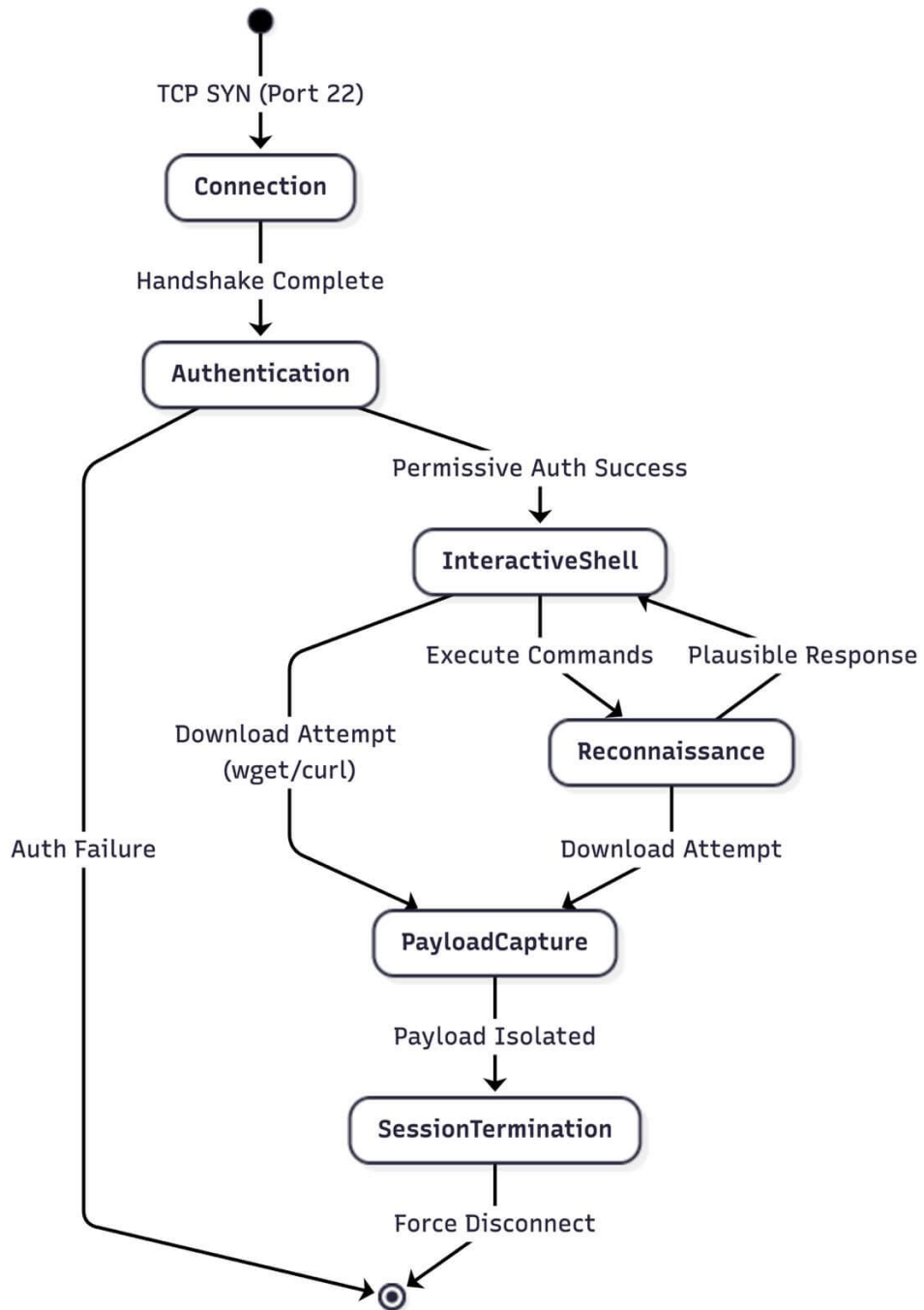


Figure 3.2: Finite State Machine Diagram of Attacker Interaction

illusion of legitimacy. This phase is particularly important for encouraging automated malware scripts to continue execution instead of prematurely terminating due to environment detection.

- **Payload Capture State:** If the attacker attempts to retrieve external binaries using utilities such as `wget` or `curl`, the system transitions into a payload capture phase. The requested file is safely intercepted, downloaded, and isolated inside a secure quarantine environment for later analysis. Once the interaction concludes, the session is terminated to preserve system integrity.

3.4 Data Analysis Stack (ELK Pipeline)

Capturing malicious activity represents only the first stage of the deployment process. The raw telemetry generated by the honeypots must be transformed into structured and actionable threat intelligence. This is achieved through the implementation of the ELK data processing pipeline, composed of three integrated components:

- **Logstash (Ingestion and Enrichment):** Logstash functions as the primary data ingestion and parsing engine. It continuously consumes the raw JSON logs generated by Cowrie and Dionaea, normalizing them into a unified structure. In addition to event parsing, Logstash enriches the telemetry using GeoIP databases, appending metadata such as country, city, and Autonomous System Number (ASN) information associated with attacker IP addresses.
- **Elasticsearch (Storage and Indexing):** Elasticsearch acts as the centralized storage and indexing platform. It stores the normalized event stream using scalable time-series indexing strategies, enabling rapid querying, event correlation, and efficient management of large datasets.
- **Kibana (Visualization and Correlation):** Kibana provides the visualization and analysis interface for the infrastructure. Through dynamic dashboards, it enables real-time monitoring of attack origins, targeted protocols, authentication trends, and the most frequently observed credential combinations.

To improve resilience and preserve data integrity, periodic archival routines are integrated into the workflow, securely transferring collected telemetry outside the honeypot environment for long-term storage.

3.5 Institutional Deployment and Network Considerations

The system is hosted within the IST network infrastructure, which enforces strict filtering at the campus edge. This network environment effectively divides the deployment into two distinct operational scopes:

- **Public Facing Services:** Traffic directed at ports such as 22 (SSH), 80 (HTTP), 443 (HTTPS), 3306 (MySQL), 5432 (PostgreSQL), and 8080 (HTTP-Alt) is allowed through the institutional fire-

wall. This exposure enables the honeypots to act as external sensors, capturing global botnet activity and automated web and database exploits.

- **Restricted Services:** Several other services considered during the project, including Telnet (port 23), SMB (port 445), MSSQL (port 1433), and RDP (port 3389), were restricted by the institutional firewall and therefore inaccessible from external networks. As a consequence, these protocols could not effectively capture public attack traffic and were limited to internal or local network experimentation scenarios.

This architectural adjustment allows the infrastructure to gather threat intelligence from both global external actors and internal network vectors.

Table 3.1: Diagnostic of external port accessibility and IST firewall behavior.

Service (Port)	Result	Diagnosis	Active Component
SSH (22)	Succeeded	Open and functional	Cowrie
HTTP (80)	Succeeded	Open and functional	Dionaea
HTTPS (443)	Succeeded	Open and functional	Dionaea
MySQL (3306)	Succeeded	Open and functional	Dionaea
Postgres (5432)	Succeeded	Open and functional	Socat
HTTP-Alt (8080)	Succeeded	Open and functional	Socat
Telnet (23)	Refused	Actively rejected by the IST firewall	Cowrie
MSSQL (1433)	Refused	Actively rejected by the IST firewall	Socat
RDP (3389)	Refused	Actively rejected by the IST firewall	Socat
NetBIOS (139)	Timeout	Packets silently dropped by the IST firewall	Dionaea
SMB (445)	Timeout	Packets silently dropped by the IST firewall	Dionaea

The experimental validation of the network exposure revealed that the institutional firewall applied different filtering policies depending on the targeted service. Public-facing protocols remained externally accessible, allowing the honeypot infrastructure to capture real attack traffic.

In contrast, several administrative and database-related services were either actively rejected or silently dropped before reaching the honeypot environment. Telnet (23), MSSQL (1433), and RDP (3389) generated active rejection responses, while SMB (445) and NetBIOS (139) traffic was silently discarded by the firewall.

These restrictions directly influenced the final architecture of the system, forcing the deployment to prioritize externally accessible services while limiting certain protocols to local experimentation scenarios. This scenario also provided valuable insight into the practical challenges associated with deploying deception infrastructures inside restricted enterprise or academic network environments.

Furthermore, this network setup introduces a dual-monitoring capability. While public-facing ports act as external sensors for global botnet activity, the restricted protocols remain completely visible to the internal campus network. This architectural positioning allows the infrastructure to intercept lateral movement, internal scanning, and automated propagation attempts originating from compromised de-

vices within the institutional perimeter. Consequently, the deployment functions not only as an external threat detector but also as a hybrid platform capable of identifying internal security incidents.

3.6 Threat Intelligence Integration

To automate the analysis of malicious payloads captured during the interaction lifecycle, the architecture integrates the VirusTotal API.

Whenever the honeypot intercepts a downloaded file, a background process calculates the binary's SHA-256 cryptographic hash and automatically submits it to the external threat intelligence platform.

This integration allows the infrastructure to classify malicious files, identify known malware families, and correlate detections across multiple antivirus engines. As a result, the collected telemetry is enriched with external threat intelligence, significantly reducing the operational overhead associated with manual malware analysis while improving the overall contextualization of captured attacks.

Chapter 4

Implementation

This chapter details the practical implementation and engineering decisions behind the honeypot infrastructure proposed in Chapter 3. While the previous chapter presented the architectural design, this section focuses on the deployment process, system configuration, and the technical challenges encountered throughout the project. Specifically, it explores the containerized deployment of Cowrie, Dionaea, and Socat, the Network Address Translation (NAT) mechanisms used for service exposure, the HoneyFS customizations applied to improve deception realism, and the integration of automated malware analysis workflows. Finally, Section 4.7 discusses the operational trade-offs and deployment limitations imposed by institutional network restrictions and honeypot fingerprinting techniques.

4.1 Deployment Environment and Initial Configuration

The core infrastructure was deployed on an Ubuntu Linux virtual server, provisioned within an Open-Stack infrastructure hosted on the Instituto Superior Técnico (IST) campus network. This environment provided a stable and centralized foundation for the deception ecosystem.

The implementation phase began with the deployment of the Cowrie honeypot. To comply with the principle of least privilege and minimize the risk of host compromise, the service was configured to run under a dedicated unprivileged user account. This ensured that even if a vulnerability within the honeypot software were exploited, the attacker would remain isolated from the underlying host operating system.

The initial configuration phase established the baseline infrastructure, including internal networking parameters, isolated logging directories, filesystem organization, and the execution boundaries required for safe public exposure.

4.2 SSH Traffic Redirection and Service Exposure

Exposing an interactive terminal environment to the public internet presents a significant architectural challenge: balancing effective threat capture with infrastructure security. By default, automated botnets

systematically target SSH services running on port 22. However, binding a honeypot directly to the host's primary administrative interface introduces severe operational conflicts and security risks. To address this issue, a structured network isolation and port-forwarding strategy was implemented.

First, the host operating system's legitimate SSH daemon, used exclusively for administrative maintenance, was reassigned to a non-standard high-numbered port (2225). This reconfiguration protected the real administrative interface from the continuous volume of automated malicious traffic.

Subsequently, Cowrie was deployed inside an isolated Docker container configured to listen internally on port 2222. To expose the decoy service externally, Docker's native iptables-based NAT port-mapping mechanism was used to transparently forward incoming traffic from external port 22 to the container's internal port 2222.

This implementation successfully validated the system's external visibility, enabling the collection of real-world telemetry while preserving host integrity and operational isolation.

4.3 Cowrie Customization and HoneyFS Enhancements

To maximize the quality of the collected threat intelligence, the simulated environment needed to exhibit a high degree of realism. Advanced malware and experienced attackers frequently rely on fingerprinting techniques to identify sandboxed or emulated systems. Once anomalies are detected, interactions are often terminated immediately. Consequently, several modifications were applied to Cowrie's simulated filesystem (HoneyFS) to reduce fingerprinting indicators and improve attacker engagement.

One of the primary modifications involved replacing Cowrie's default hostname (`svr04`) with an enterprise-style identifier, creating the appearance of a production database server. Additionally, the authentication policy defined within the `userdb.txt` configuration file was intentionally configured to be permissive. This allowed the `root` account to authenticate successfully regardless of the submitted password, maximizing the probability of observing post-authentication activity instead of merely collecting failed login attempts.

After authentication, attackers were presented with a customized filesystem populated with strategically placed digital decoys. Simulated configuration files (`.env`, `config.db`), cryptographic materials (`id_rsa` keys), and credential-related files (`passwords.txt`) were distributed across standard Linux directories to encourage exploration and interaction.

Furthermore, specific command responses were refined to generate plausible outputs during automated reconnaissance. To achieve this, Cowrie's simulated output templates were modified to alter the results of hardware and operating system profiling commands. For example, queries frequently used by automated scripts, such as `cat /proc/cpuinfo`, `df -h`, and `uname -a`, were configured to return realistic processor core counts, plausible disk space allocations, and kernel version information consistent with a production Ubuntu environment, rather than exposing default honeypot artifacts. These modifications increased session duration and encouraged attackers to perform additional actions, allowing the infrastructure to capture richer telemetry related to system reconnaissance, privilege validation, and persistence-oriented behavior.

4.4 Dionaea Deployment and Multi-Service Expansion

Following the stabilization of the terminal-based honeypot, the infrastructure was expanded to capture application-layer and network-oriented exploits through the integration of Dionaea. The original objective was to monitor SMB-related attacks due to the protocol's importance in ransomware propagation and lateral movement.

However, practical deployment confirmed that the institutional firewall enforced strict ingress filtering on SMB traffic. To adapt to this limitation, the Dionaea configuration was strategically redirected toward services permitted through the campus perimeter. The deployment was therefore reoriented to monitor HTTP (port 80), HTTPS (port 443), and MySQL (port 3306).

To further expand the observable attack surface without the overhead associated with deploying full application servers, Socat was introduced as a lightweight relay component. Acting as a low-interaction service layer, Socat exposed PostgreSQL (port 5432) and HTTP-Alt (port 8080), enabling the infrastructure to capture automated scanning and service enumeration attempts.

Through strict containerized port segregation, Cowrie, Dionaea, and Socat coexisted successfully on the same host without binding conflicts, effectively compartmentalizing the attack surface across distinct protocol vectors.

4.5 Malware Capture

Beyond recording interactive commands, one of the main implementation objectives involved the safe acquisition and classification of malicious payloads. Cowrie was configured to intercept file retrieval attempts, particularly those executed through utilities such as `wget` and `curl`. Instead of allowing these binaries to execute within the environment, the honeypot intercepted the requests, downloaded the payloads, and quarantined them within a secure isolated directory.

To automate the malware classification process and reduce the need for manual reverse engineering, a custom script was integrated into the deployment workflow using the VirusTotal API [22]. Whenever a new binary was introduced into the quarantine directory, the script automatically generated its SHA-256 cryptographic hash and submitted it to the VirusTotal platform.

The retrieved classification reports were subsequently correlated with the corresponding session logs, enabling the immediate identification of malware families and enriching the local telemetry with external threat intelligence.

4.6 Logging, Monitoring, and Data Visualization

Managing the substantial volume of telemetry generated by the multi-honeypot deployment required a robust and structured data analysis pipeline. Both Cowrie and Dionaea were configured to export activity logs natively in JSON format, enabling centralized processing and event correlation.

These logs were continuously ingested through Logstash pipelines monitoring the honeypot output directories. Logstash parsed the incoming events and applied GeoIP enrichment filters to append geographical metadata to incoming IP addresses. The processed datasets were then indexed within Elasticsearch using daily index partitioning to optimize long-term querying and correlation performance.

At the visualization layer, custom Kibana dashboards were developed to provide real-time operational awareness. These dashboards aggregated the collected telemetry into specific visual indicators, including geographic attack maps to track source IP origins, protocol distribution charts to monitor targeted ports (e.g., 22 vs 80), credential exploitation statistics highlighting the most attempted passwords, and malware activity indicators tracking payload hashes.

Finally, to ensure data persistence and infrastructure resilience, automated cron-based backup routines were implemented to periodically compress and transfer log archives to secure off-host storage.

4.7 Challenges and Deployment Limitations

The implementation of this infrastructure encountered several practical engineering challenges, primarily related to institutional network constraints and the adaptive nature of modern cyber threats.

The most significant limitation originated from the strict ingress filtering enforced by the campus firewall. The active rejection or silent dropping of protocols such as Telnet, SMB, RDP, and MSSQL substantially restricted the system's ability to capture certain categories of malware propagation and remote exploitation attempts. Nevertheless, rather than undermining the project, these restrictions provided a realistic case study of deploying active defense mechanisms within heavily regulated enterprise-like environments, ultimately forcing the architecture to prioritize highly accessible SSH and web-oriented attack vectors.

A secondary challenge involved balancing deception realism with host security. Despite the extensive HoneyFS customizations, including the deployment of enterprise-style hostnames, decoy configuration files, simulated cryptographic artifacts, and tailored command outputs, advanced malware strains employing sophisticated fingerprinting techniques (such as validating CPU instructions, filesystem inconsistencies, virtualization artifacts, or kernel behaviors) may still identify the simulated nature of the environment. Once detection occurs, automated scripts typically terminate the interaction immediately, resulting in truncated sessions and reduced post-exploitation telemetry.

Balancing the external exposure required for effective threat intelligence collection while simultaneously maintaining strict containment boundaries proved to be one of the defining engineering challenges of the entire deployment.

4.8 Security and Containment Measures

Operating an active cyber defense infrastructure requires strict containment and access control measures to protect the host system and ensure that malicious activity remains fully isolated.

At the system and application level, containment was achieved through containerized virtualization. As validated during deployment, all primary honeypot services (Cowrie, Dionaea, and the Socat relays), were deployed inside isolated Docker containers. This architecture successfully decoupled the exposed and vulnerable services from the host's core operating system. Because these containers operate as medium and low-interaction environments, commands executed by attackers are intercepted and handled by internal emulations. This mechanism, combined with the containerized boundaries, prevents the execution of real, destructive actions and restricts unauthorized outbound traffic from being generated by the host server.

Furthermore, strict administrative access controls were enforced to protect the collected threat intelligence. The backend components, including the Elasticsearch database and the Kibana visualization dashboard, were also containerized. To protect these sensitive interfaces from unauthorized public access, the research team interacted with the Kibana dashboard exclusively through an encrypted SSH local port forwarding tunnel. To establish this secure connection, authentication was restricted to a non-standard administrative SSH port and required cryptographic identity verification via a dedicated `.pem` private key. This multi-layered access policy successfully prevented unauthorized external access to the infrastructure's monitoring backend, ensuring the integrity and confidentiality of the collected data.

Chapter 5

Final Tests and Results

Rather than simply presenting raw telemetry, this chapter analyzes the collected dataset to identify recurring patterns of automated scanning, brute-force authentication attempts, malicious payload delivery attempts, and post-authentication behavior. Section 5.1 describes the experimental methodology and provides a global overview of the captured traffic throughout the monitoring period. Section 5.2 examines the attack surface by analyzing the distribution of interactions across the deployed services and honeypot components. Section 5.3 profiles the observed threat activity through geolocation analysis and source distribution patterns. Section 5.4 investigates credential exploitation campaigns targeting the SSH service, focusing on brute-force authentication attempts and commonly used credentials. Finally, Section 5.5 analyzes post-authentication attacker behavior, including reconnaissance activities, persistence-related actions, and command execution patterns observed within the interactive honeypot environment.

5.1 Experimental Methodology and Global Traffic Overview

Before analyzing specific threat vectors, it is necessary to define the scope and structure of the experimental dataset. The honeypot infrastructure remained publicly exposed throughout the monitoring period, continuously capturing interactions across all deployed components, namely Cowrie, Dionaea, and Socat. To preserve analytical integrity, the dataset was filtered to exclude traffic related to internal routing behavior, local testing procedures, and routine institutional scanning activity.

Events were categorized into two main classes:

- **Network-layer events:** including TCP connection attempts and port probes captured through NAT forwarding and firewall telemetry.
- **Application-layer events:** including SSH authentication attempts, command execution activity, malware download requests, and service-specific interactions recorded directly by the honeypots.

For the purposes of this study, an event is therefore defined as any distinct interaction successfully captured either at the network or application layer during the deployment period.

Table 5.1: Global Experimental Dataset Metrics

Category	Metric	Value
Temporal	Total Monitoring Period	84 days
Temporal	Peak Daily Traffic	145,979 Events (April 16)
Volume	Total Captured Events	583,521
Volume	Unique Source IPs	2,966
Volume	Average Events per Hour	~289
Service Exposure	Most Targeted Web Service	HTTP (Port 80)
Service Exposure	Most targeted Interactive Service	SSH (Port 22/2222)
Malware Analysis	Captured Malware Payloads	0

As presented in Table 5.1, no malicious payloads were successfully captured during the monitoring period, despite the integration of the VirusTotal analysis pipeline. This outcome does not indicate a failure of the collection infrastructure, but instead reflects the specific behavioral characteristics of the observed attack activity.

The majority of captured interactions were associated with automated SSH brute-force attempts, credential testing, and initial reconnaissance operations. In many cases, attackers terminated the session shortly after performing basic environment validation and system profiling commands, without proceeding to secondary payload delivery stages.

One possible explanation is related to the medium-interaction nature of the deployed HoneyFS environment. Although the infrastructure successfully simulated an interactive Linux system, more advanced automated frameworks may rely on additional environmental indicators, kernel-level behavior, or vulnerable service responses before attempting malware deployment or binary execution.

Consequently, while the infrastructure proved effective at capturing authentication activity and post-authentication behavioral patterns, the observed campaigns did not progress toward large-scale malware transfer during the analyzed period.

The relatively high ratio between total events and unique source IPs suggests repeated automated interactions rather than isolated opportunistic scans. By establishing this baseline, it becomes possible to understand both the scale and persistence of the captured activity. Figure 5.1 demonstrates the temporal distribution of attack events throughout the deployment period.

The temporal distribution shows two clearly different traffic patterns: a persistent baseline of low-interaction reconnaissance activity and a sudden burst of high-frequency attack traffic.

The persistent baseline activity suggests that publicly exposed infrastructure remains under continuous automated scanning, independently of organizational profile or service criticality. This behavior is consistent with the observations described by Provos et al. [2], who identified internet-wide scanning as a persistent characteristic of automated cyber threats.

However, the most significant anomaly occurred on April 16th, when the infrastructure recorded 145,979 events within a single 24-hour period. Log analysis showed that this spike was exclusively related to an aggressive SSH brute-force campaign directed at the Cowrie honeypot. During this specific period, the Dionaea services recorded zero interactions, while port 22 received 100% of the attack volume.

Temporal Distribution of Attack Events

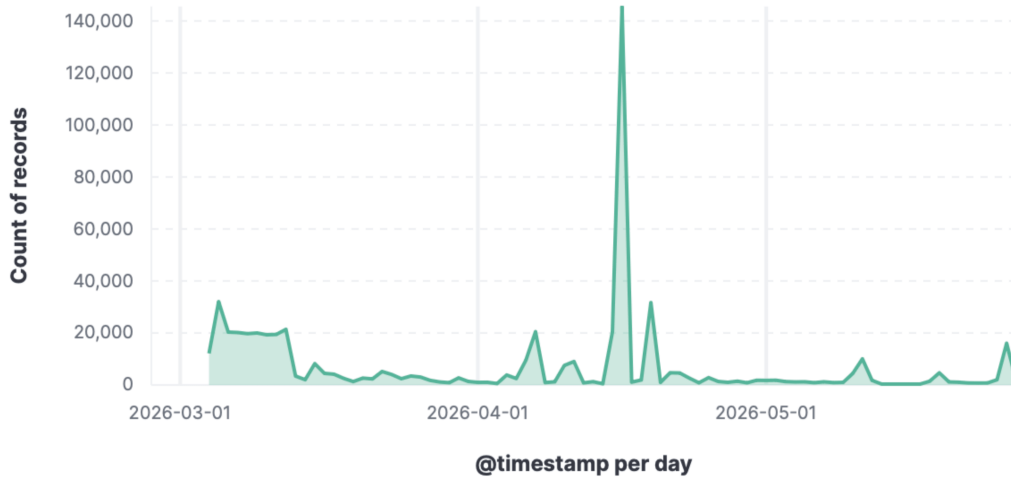


Figure 5.1: Temporal Distribution of Attack Events

Further analysis indicated that this campaign was highly concentrated, with 98.75% of the traffic originating from IP addresses geolocated to Malaysia. Unlike slower human-driven intrusion attempts, the observed behavior showed characteristics commonly linked to automated botnet campaigns, namely very high request frequency, rapid scanning patterns, and the immediate execution of automated post-authentication verification scripts (which will be discussed further in Section 5.5).

Similar behaviors were documented by Antonakakis et al. [14], who demonstrated that compromised botnet nodes frequently perform highly parallel scanning operations targeting vulnerabilities across large IP ranges.

The rapid increase and later drop in activity observed during this deployment follows this operational model, suggesting that the infrastructure was temporarily targeted by a large automated campaign before the botnet moved its activity toward different network ranges.

5.2 Attack Surface and Targeted Services

Evaluating the distribution of targeted services provides useful information about the priorities of automated threat actors. Building on the event classification established in Section 5.1, the attack surface analysis is divided between application-layer interactions (recorded directly by Dionaea and Cowrie) and network-layer probes (obtained from firewall NAT rules for the Socat decoys). This separation allows a clearer understanding of how different protocols attract different attack vectors.

5.2.1 Web and Database Probes (Dionaea)

The Dionaea honeypot was configured to monitor medium-interaction application-layer services, specifically HTTP (port 80), HTTPS (port 443), and MySQL (port 3306). Figure 5.2 illustrates the distribution of captured interactions across these exposed services.

Target Port Distribution (Dionaea Honeypot)

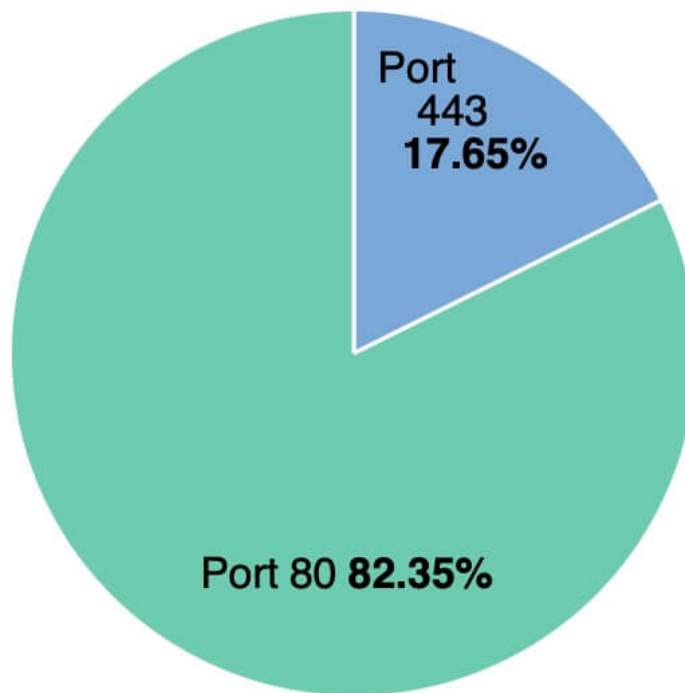


Figure 5.2: Distribution of Attacked Network Services (Dionaea)

The dataset reveals a clear imbalance in the distribution of targeted services. HTTP traffic on port 80 accounted for 82.35% of all captured interactions, while HTTPS traffic on port 443 represented the remaining 17.65%. In contrast, the MySQL service (port 3306) registered no significant activity within this specific application-layer dataset.

This absence of MySQL-related traffic is primarily explained by deployment timing and host-level configuration constraints. During the initial baseline collection phase, port 3306 was not yet fully provisioned within the honeypot infrastructure and therefore remained unavailable for external interaction. Although the service was later enabled manually on the host system to validate operational readiness, the exposure period was relatively short and insufficient to capture meaningful volumes of automated database-oriented probing within the analyzed logs.

The resulting traffic profile differs considerably from honeypot deployments operating in unrestricted environments. As reported by Hussain et al. [12], Dionaea instances exposed in public cloud infrastructures commonly capture substantial volumes of Server Message Block (SMB) exploitation attempts and malware propagation activity. In the present deployment, however, inbound SMB traffic was filtered by the institutional firewall, causing the observable attack surface to shift almost entirely toward web-oriented services.

Furthermore, the strong predominance of HTTP traffic over HTTPS reflects behavior typically associated with large-scale automated reconnaissance campaigns. Automated scanning frameworks generally prioritize speed, scalability, and low connection overhead. Since HTTP does not require TLS negotiation, attackers can generate requests more rapidly and with lower computational cost, making unencrypted

web services particularly attractive targets for high-frequency internet-wide scanning activity.

5.2.2 Port Scanning and Low-Interaction Decoys (Socat)

To capture reconnaissance activity directed at services beyond the scope of Cowrie and Dionaea, Socat relays were deployed strictly as transport-layer decoys. These instances were configured only to complete the TCP handshake, simulating open administrative and database ports without exposing functional application-layer services.

Because this architectural approach does not generate application telemetry, the interaction volume was measured by analyzing the host iptables NAT forwarding rules. This method provides accurate packet and byte counts for the network-layer probes redirected to each isolated Docker container.

Table 5.2 presents the recorded interactions for these supplementary services.

Table 5.2: Network-Layer Traffic Metrics for Socat Decoys

Service	Port	Packets Received	Data Volume (Bytes)
HTTP-Alt	8080	1,199	63,563
PostgreSQL	5432	564	32,448
RDP	3389	5	320
MSSQL	1433	1	64

The firewall metrics confirm both the network exposure conditions and the priorities of automated reconnaissance activity. Externally accessible services registered continuous probing throughout the deployment period.

The traffic concentration observed on HTTP-Alt (port 8080) is consistent with automated discovery activity targeting secondary web services, development environments, and administrative interfaces commonly found in web infrastructures. Similarly, the activity recorded on port 5432 suggests automated database enumeration attempts.

In contrast, services restricted by the institutional firewall, specifically Remote Desktop Protocol (port 3389) and Microsoft SQL Server (port 1433), recorded almost no interaction. The small footprint of 5 and 1 packets, respectively, is more likely related to internal routing behavior than external threat activity.

These observations confirm the effectiveness of the ingress filtering controls while highlighting how real-world network topology constraints influence honeypot telemetry collection.

5.2.3 Terminal Services (Cowrie)

The Cowrie honeypot was specifically deployed to capture interactive terminal sessions and authentication-based exploitation attempts. Figure 5.3 illustrates the protocol distribution for this architectural component.

The dataset shows that traffic directed at port 22 (SSH) represented the dominant target service within this environment, accounting for 93.15% of total interactions. The remaining 6.81% targeted port 53 (DNS).

Target Port Distribution (Cowrie Honeypot)

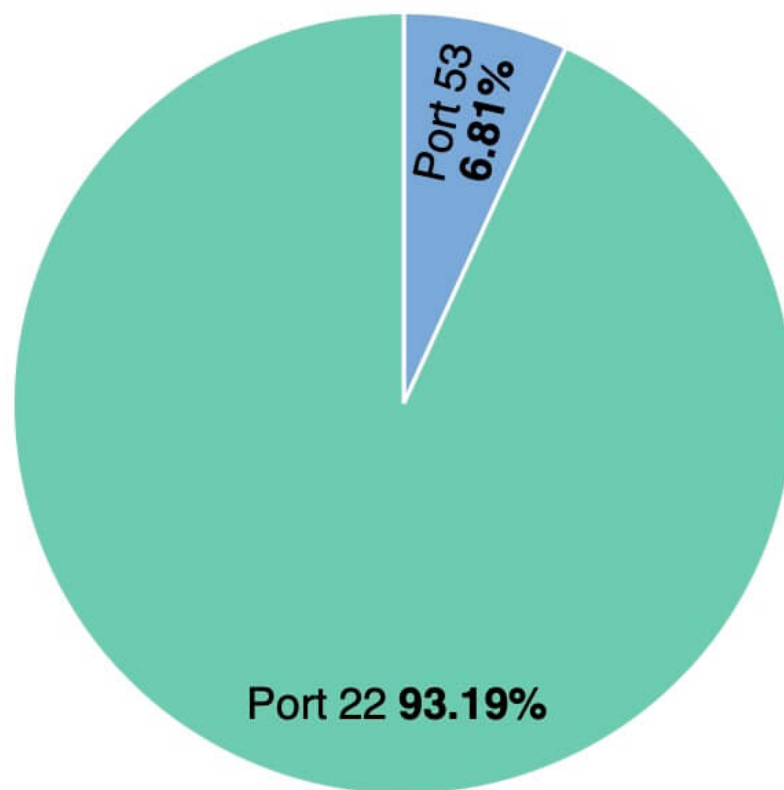


Figure 5.3: Distribution of Attacked Network Services (Cowrie)

The high volume of SSH activity is consistent with continuous internet-wide brute-force campaigns. As previously observed during the April 16th anomaly, automated botnets frequently target remote administration services as part of initial access workflows.

Furthermore, the successful capture of this traffic without affecting the host validates the Docker port-mapping strategy discussed in Chapter 4. By redirecting traffic originating from the default SSH port into the isolated container environment, the architecture successfully captured authentication attempts while keeping the host administrative interface separated.

Regarding the residual activity observed on port 53, although Cowrie does not emulate a fully functional DNS resolver, these interactions still provide useful visibility into reconnaissance activity. The observed traffic may reflect automated discovery attempts targeting exposed DNS services or resolver configurations.

Overall, these observations indicate that terminal-related attack activity was successfully isolated and monitored, confirming the effectiveness of both the hybrid deployment model and the ingress filtering strategy adopted by the infrastructure.

5.3 Threat Profiling and Geolocation

Beyond analyzing the targeted attack surface, evaluating the origin and distribution of incoming traffic provides additional information about automated scanning behavior. Using the Logstash GeoIP enrichment pipeline, the infrastructure mapped the source IP addresses of captured events, allowing the creation of a behavioral profile of the observed campaigns.

5.3.1 Geographical Distribution

Figure 5.4 presents a treemap showing the geographical origin of the captured interactions.

The dataset shows a global distribution of source traffic, although with noticeable regional concentration patterns. Malaysia emerged as the main source, accounting for 44.49% of all recorded interactions, followed by Germany (8.38%), the United States (6.65%), China (5.08%), Poland (4.93%), and Russia (3.81%).

The strong concentration observed in Malaysia is mainly related to the April 16th anomaly discussed in Section 5.1. Log analysis confirmed that most of this traffic was generated during that highly focused SSH brute-force campaign.

While global botnet traffic dominated the overall dataset, analyzing localized threat activity provides important context regarding the domestic and institutional threat landscape. Traffic originating from Portugal accounted for a minimal fraction of the total volume, representing just 0.16% with 710 captured events. Further filtering of these domestic interactions revealed that zero (0) events originated directly from within the Instituto Superior Técnico (IST) network perimeter (193.136.0.0/16). This empirical finding is highly relevant: despite the honeypots being fully exposed to the internal campus network, the absence of localized probing suggests effective internal network segmentation or a lack of actively

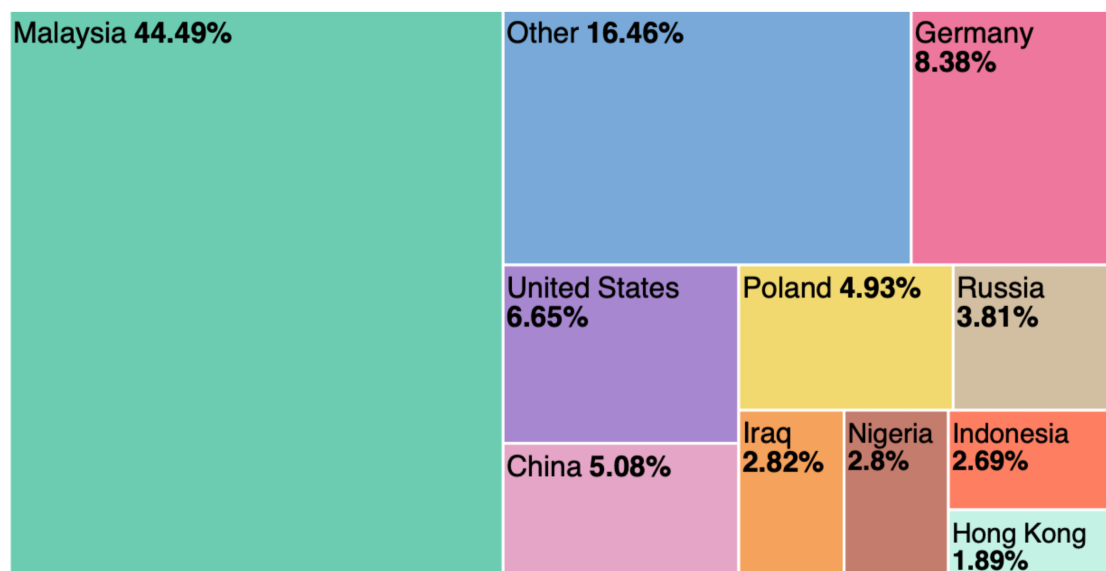


Figure 5.4: Geographical Origin of Attacks

scanning compromised devices within the institutional perimeter during the monitoring period.

However, from a threat intelligence perspective, these geographical results should not be interpreted as the real location or nationality of the original threat actors. These locations more likely represent intermediary infrastructure such as compromised servers, VPN endpoints, and vulnerable IoT devices integrated into distributed networks.

5.3.2 Threat Actor Decentralization

Profiling individual source IP addresses provides additional context regarding the observed campaigns. Figure 5.5 illustrates the source IP addresses responsible for the highest interaction volume.

Among the identifiable contributors, the most active unique IP address (3.81.115.206) recorded approximately 90 interactions. Threat intelligence enrichment revealed that this address belongs to an Amazon Web Services (AWS) data center located in the United States, suggesting the use of a compromised cloud instance or an ephemeral virtual machine to conduct automated reconnaissance. While this individual number appears very small compared to the total volume of 583,521 events, it highlights the distributed nature of the automated campaigns.

The large-scale attack activity observed during the monitoring period, particularly the traffic concentration associated with Malaysia, did not originate from a small number of highly active hosts. Instead, the botnet relied on thousands of compromised devices, where each individual node performed only a limited number of connection attempts before rotating to another source IP address.

This behavior explains why no individual IP address dominates the chart and why the “Other” category accounts for the vast majority of the distribution, as most of the brute-force activity was dispersed across a highly decentralized pool of transient nodes.

Top Aggressive Source IP Addresses

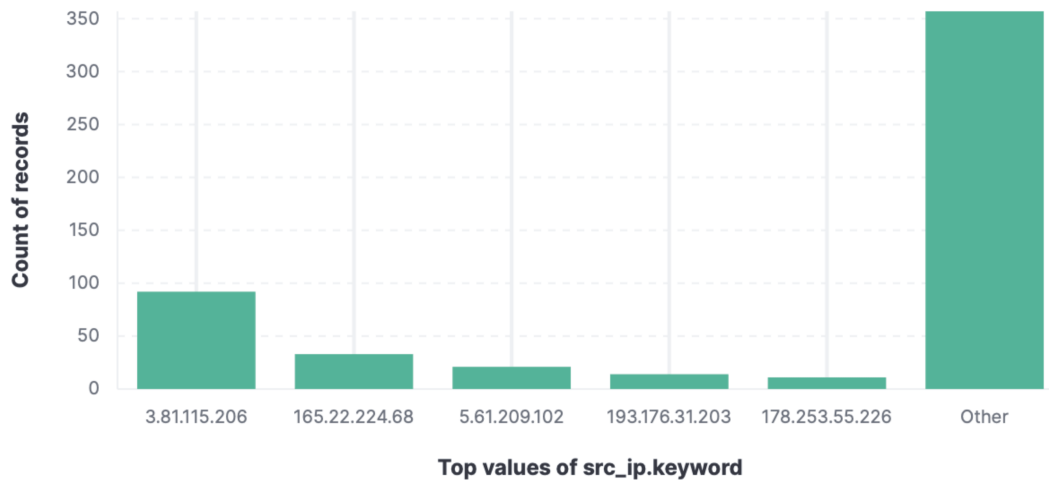


Figure 5.5: Top Source IP Addresses

This dispersion pattern is consistent with decentralized automated scanning activity frequently associated with botnet infrastructures. The observed traffic distribution may reflect the use of large pools of intermediary systems and rotating source addresses, behaviors commonly reported in large-scale automated campaigns.

These observations reinforce the distributed nature of the captured threat activity while also suggesting limitations in purely static IP-based filtering approaches.

5.4 Credential Exploitation (Brute-Force Analysis)

To gain initial access to the infrastructure, automated attackers continuously performed dictionary based brute force attempts against the exposed SSH service. By analyzing the authentication attempts captured by the Cowrie honeypot, it becomes possible to identify the usernames and passwords most frequently used throughout these campaigns.

Figure 5.6 illustrates the distribution of the most attempted credentials observed during the deployment period.

The collected data reveals a strong reliance on automated credential testing, particularly targeting common administrative accounts and weak default passwords. To provide a clearer perspective on the attacker priorities and to facilitate comparison with the web deception infrastructure deployed by Braz [16], the most frequently observed authentication attempts can be summarized in Table 5.3.

This distribution highlights one of the most notable patterns: the overwhelmingly high volume of authentication attempts using empty passwords. This behavior is commonly associated with automated scanning tools searching for misconfigured systems or devices that allow authentication without proper password enforcement.

Comparing these results with the baseline study by Braz [16] reveals both structural similarities and

Most Attempted Brute-Force Credentials

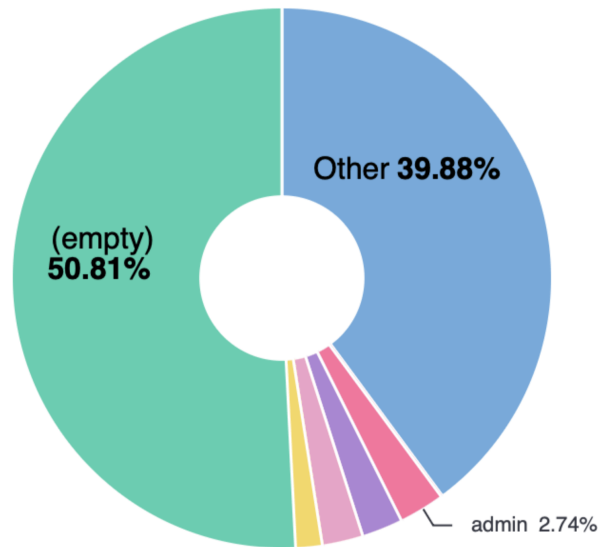


Figure 5.6: Distribution of Most Attempted Brute-Force Credentials

Table 5.3: Top Observed Authentication Attempts

Username	Password	Percentage
[Valid Username]	(Empty Password)	50.81%
admin	[Password]	2.74%
root	[Password]	0.10%
user	[Password]	0.04%

protocol specific differences. In the WordPress honeypot developed by Braz, the password "admin" was overwhelmingly the most attempted credential, driven by automated scripts exploiting the XML-RPC functionality. While our SSH deployment observed a much higher prevalence of empty passwords, both studies confirm a universal reliance on static dictionary lists. Combinations utilizing usernames like root and user, paired with weak passwords like 123456 and admin123, populated the top ranks of both datasets.

The remaining 46.31% of the interactions were distributed across a large number of less frequent credential combinations. This broad distribution reflects the behavior of automated botnets that continuously rotate through extensive credential dictionaries in an attempt to compromise exposed services.

Because the Cowrie honeypot was intentionally configured with a permissive authentication policy, the infrastructure successfully captured not only initial access attempts but also the actions performed by attackers after login. This approach provided additional visibility into the operational behavior of automated intrusion scripts beyond the initial authentication phase.

5.5 Post-Authentication Behavior and Command Analysis

Following successful authentication through Cowrie's permissive credential policy, attackers were granted access to the simulated HoneyFS environment. Analysis of the captured sessions shows that most interactions followed a predictable sequence of actions, suggesting the use of automated scripts rather than human-operated attacks.

The primary vulnerability exploited during this initial access phase is weak authentication. Rather than using software exploits, the automated scripts rely on the widespread vulnerability of systems configured with default, easily guessable, or blank passwords.

Table 5.4 summarizes the most frequently executed commands captured by the Cowrie honeypot, together with their primary operational objectives.

Table 5.4: Top Executed Commands Captured within the Interactive Shell

Command String	Execution Count	Primary Operational Objective
echo -e "\x6F\x6B"	26,161	Automated Session Verification & Obfuscation
cd ~; chatter -ia .ssh; lockr -ia .ssh	3,126	Persistence Execution & Directory Unlocking
lockr -ia .ssh	3,126	Non-Standard Permission Manipulation
uname -a	3,074	Kernel & OS Architecture Reconnaissance
uname -s -v -n -r -m	1,077	Detailed OS Fingerprinting
cat /proc/cpuinfo grep name head -n 1 awk '{print \$4,\$5,\$6,\$7,\$8,\$9;}'	144	Hardware Core Count Assessment
whoami	140	Privilege Level Verification
df -h head -n 2 awk 'FNR == 2 {print \$2;}'	139	Storage and Volatile Memory Profiling
rm -rf /tmp/secure.sh; rm -rf /tmp/auth.sh; pkill -9 secure.sh; pkill -9 auth.sh; echo > /etc/hosts.deny; pkill -9 sleep;	139	Anti-Forensics and Trace Wiping
crontab -l	139	Scheduled Task Enumeration (Persistence)

The empirical data indicates that post-authentication activity was dominated by four main operational phases.

Session Verification The most frequently executed command was `echo -e "\x6F\x6B"`, with a total of 26,161 executions. The `-e` flag enables the shell to interpret escaped hexadecimal sequences, which in this case decode to the ASCII string "ok".

This command appears to serve as a simple verification mechanism used by automated attack scripts. Before downloading additional payloads or executing further actions, the script confirms that the remote shell is active and capable of processing commands correctly. The use of hexadecimal notation instead of plain text is a common scripting practice and may also help avoid basic pattern-matching mechanisms that search for specific strings in network traffic or command logs.

Privilege Validation and Persistence Preparation After gaining shell access, many sessions executed the `whoami` command (140 instances), likely to determine the privileges associated with the current user. This type of verification is commonly used after authentication to confirm the level of access available before proceeding with additional actions.

Several sessions also interacted with the `.ssh` directory through commands such as `cd ~; chattr -ia .ssh; lockr -ia .ssh` and the standalone variant `lockr -ia .ssh` (3,126 executions each). The native `chattr` command, combined with the `-ia` flags, attempts to remove the Immutable (i) and Append-only (a) filesystem attributes from files or directories. These attributes are commonly used to restrict modifications to sensitive resources.

By attempting to alter these immutable attributes, the automated scripts are attempting to exploit a common configuration vulnerability: excessive user privileges. The attack assumes that the compromised account has been misconfigured with sufficient administrative rights to bypass standard file protections and establish persistent access.

The observed command sequence suggests an attempt to modify the contents of the `.ssh` directory, potentially to add SSH keys or otherwise alter authentication-related files. The repeated appearance of the non-standard utility `lockr` may indicate that these commands originated from pre-packaged scripts or malware toolkits, although its exact purpose could not be verified from the collected data.

Additionally, the execution of `crontab -l` (139 instances) indicates that attackers were inspecting scheduled tasks configured on the system. Such activity may be associated with reconnaissance, the verification of existing persistence mechanisms, or the preparation of additional automated tasks.

System Reconnaissance and Resource Profiling Once shell access was established, many sessions performed basic system reconnaissance to gather information about the host environment. Commands such as `uname -a` (3,074 executions) and `uname -s -v -n -r -m` (1,077 executions) were frequently used to retrieve details about the operating system, kernel version, hostname, and processor

architecture. Collecting this information is a common step in automated attacks, as it helps determine the characteristics of the target system before further actions are performed.

Resource availability was also assessed through commands that queried CPU information and disk capacity. Examples include `cat /proc/cpuinfo | grep name | head -n 1 | awk '{print $4,$5,$6,$7,$8,$9;}'` (144 executions) and `df -h | head -n 2 | awk 'FNR == 2 {print $2;}'` (139 executions). These commands extract information about the processor model and available storage resources.

The repeated execution of these reconnaissance commands suggests that many of the observed attacks followed a structured workflow in which information about the host was collected before attempting additional actions. Such information could be used to assess whether the system is suitable for running specific payloads or supporting resource-intensive activities, such as cryptocurrency mining, proxy services, or participation in distributed denial-of-service (DDoS) botnets.

Trace Removal The final stage observed in several sessions involved command sequences aimed at removing temporary files and terminating associated processes. A representative example is shown below:

```
rm -rf /tmp/secure.sh; rm -rf /tmp/auth.sh; pkill -9 secure.sh;
pkill -9 auth.sh; echo > /etc/hosts.deny; pkill -9 sleep;
```

These commands attempt to delete temporary scripts stored in the `/tmp/` directory and terminate any running instances related to them. The use of `echo > /etc/hosts.deny` also suggests an attempt to modify local access-control settings by clearing the contents of the file.

Modifying critical network files like `/etc/hosts.deny` targets the vulnerability of improper access control. The scripts assume the compromised system suffers from a lack of privilege separation, allowing a standard user account to overwrite system-wide security policies.

Although these actions had no real impact on the host system due to the isolated nature of the honeypot environment, they provide insight into the behavior of the automated scripts being executed. The combination of file removal, process termination, and configuration changes suggests an effort to clean up previously executed components and prepare the system for subsequent activity.

Overall, the captured command sequences strongly indicate that the observed activity was generated by automated attack frameworks rather than human-operated sessions. The recurring pattern of session verification, privilege checking, system reconnaissance, persistence-related actions, and cleanup reflects a standardized workflow commonly associated with large-scale botnet campaigns targeting exposed SSH services.

Chapter 6

Discussion and Conclusions

This final chapter summarizes the principal findings of the project and evaluates the effectiveness of the deployed multi-honeypot infrastructure in relation to the initial research objectives. Section 6.1 discusses the conclusions derived from the collected telemetry, highlighting both the operational strengths and architectural limitations identified throughout the deployment period. Finally, Section 6.2 presents recommendations for future work, proposing technical improvements aimed at increasing environmental realism and expanding threat capture capabilities.

6.1 Discussion and Conclusions

The results collected throughout the 84-day monitoring period demonstrate that low- and medium-interaction honeypots can be effective tools for studying real-world attack activity. By recording more than 583,000 events, the deployed infrastructure generated a substantial dataset that enabled the analysis of automated reconnaissance, brute-force campaigns, malware delivery attempts, and post-authentication behavior.

One of the most significant findings of this project was the value of adopting a multi-honeypot architecture rather than relying on a single deception technology. The collected telemetry showed that different services attracted different forms of malicious activity. Cowrie proved highly effective at capturing SSH-oriented brute-force campaigns, automated command execution workflows, and post-authentication behavior. At the same time, the integration of Dionaea and Socat expanded the observable attack surface by exposing additional web and alternative network services.

Although the original objective of the Dionaea deployment was to monitor SMB-related exploitation attempts, institutional firewall restrictions prevented inbound SMB traffic from reaching the infrastructure. Rather than invalidating the experiment, this limitation provided an important practical observation: when enterprise-oriented protocols are filtered at the network perimeter, automated threat activity tends to shift toward publicly accessible services such as HTTP (port 80), HTTPS (port 443), and other exposed web interfaces.

The behavioral analysis performed after successful authentication also provided valuable insight into

the operational logic of modern automated attacks. Instead of immediately deploying destructive payloads, most observed sessions followed structured workflows involving environment verification, privilege validation, system reconnaissance, persistence preparation, and basic trace-removal procedures. These observations suggest that many contemporary botnets operate through standardized and highly automated execution chains rather than purely opportunistic interactions.

Additionally, the customization of the HoneyFS environment proved important for improving attacker engagement. The introduction of realistic hostnames, decoy configuration files, simulated SSH keys, and customized command responses reduced basic fingerprinting indicators and prolonged interaction duration. As a result, the infrastructure captured behavioral patterns that would likely remain unobserved in a default honeypot deployment.

The objectives defined at the beginning of this project were successfully achieved. The proposed architecture combined multiple honeypot technologies, centralized telemetry collection through the ELK Stack, and automated malware enrichment using VirusTotal. Together, these components enabled the collection, correlation, and analysis of attack activity across multiple services and interaction levels within a controlled environment.

In conclusion, the project successfully demonstrated the feasibility and effectiveness of a modular multi-honeypot infrastructure for threat monitoring and behavioral analysis. Despite the operational limitations imposed by the institutional environment, the platform collected a significant volume of telemetry and provided valuable insight into automated attack activity targeting exposed services. The results show that a carefully designed honeypot ecosystem, supported by centralized monitoring and malware analysis capabilities, can serve as a practical and cost-effective approach for studying contemporary cyber threats while maintaining strong isolation and security controls.

6.2 Future Work

Although the deployed infrastructure successfully captured substantial volumes of threat telemetry, the continuous evolution of cyber threats creates ongoing challenges for deception-based environments. Based on the operational limitations identified throughout this project, several technical improvements and research directions are recommended for future work.

6.2.1 Transition to High-Interaction Honeypots

One of the main limitations of medium-interaction honeypots such as Cowrie is their dependence on simulated environments. Advanced malware and sophisticated intrusion frameworks frequently employ fingerprinting techniques capable of detecting inconsistencies related to virtualization artifacts, incomplete kernel behavior, or unrealistic filesystem responses. Once the deception is detected, interactions are often terminated immediately.

A natural evolution of this project would therefore involve the deployment of high-interaction honeypots based on fully isolated virtual machines. Instead of interacting with a simulated shell environ-

ment, attackers would operate inside real operating systems deployed within strongly sandboxed environments. This approach would significantly increase environmental realism and potentially allow the capture of more advanced post-exploitation behavior, although at the cost of higher containment and operational security requirements.

6.2.2 Dynamic Cryptographic Deception

Another potential improvement involves the implementation of dynamically generated cryptographic artifacts. Because default or static SSH host keys can be used by automated scripts for honeypot fingerprinting, future iterations could introduce the periodic rotation of keys and TLS certificates. This approach would reduce signature-based detection and further increase the realism of the exposed services.

6.2.3 Advanced Malware Analysis and Sandboxing

While the current integration with VirusTotal is effective for classifying known malware families through SHA-256 hash correlation, it provides limited visibility into previously unknown or polymorphic payloads. Future work could therefore integrate automated dynamic analysis platforms such as Cuckoo Sandbox. Executing captured payloads inside isolated instrumented environments would allow researchers to extract deeper Indicators of Compromise (IoCs), including network callbacks, registry modifications, and memory injection activity.

6.2.4 Deployment Outside Institutional Network Constraints

Finally, deploying the infrastructure within a commercial cloud environment such as Amazon Web Services (AWS) or Microsoft Azure would eliminate many of the ingress filtering restrictions imposed by the institutional firewall. This would enable the controlled exposure of protocols such as SMB, RDP, Telnet, and MSSQL, allowing future studies to capture a broader range of ransomware propagation attempts, legacy botnet activity, and enterprise-oriented exploitation campaigns.

Such an expansion would provide a more complete view of modern attack surfaces while preserving the modular architecture and centralized telemetry pipeline developed throughout this project.

Bibliography

- [1] L. Spitzner. *Honeypots: Tracking Hackers*. Addison-Wesley Professional, Boston, MA, 2003.
- [2] N. Provos. A virtual honeypot framework. In *Proceedings of the 13th USENIX Security Symposium*, San Diego, CA, 2004.
- [3] Fortinet. Fortideceptor: Deception-based breach protection. Fortinet Official Website, 2026.
- [4] SentinelOne. Singularity identity active directory security. SentinelOne Official Website, 2026.
- [5] TrapX Security. Deceptiongrid. TrapX Official Website, 2026.
- [6] Cowrie Development Team. Cowrie ssh/telnet honeypot repository. <https://github.com/cowrie/cowrie>, 2026.
- [7] Dionaea Development Team. Dionaea honeypot repository. <https://github.com/Dionaea/dionaea>, 2026.
- [8] N. Provos. Honeyd - a virtual honeypot daemon. *USENIX*, 2004.
- [9] W. Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 2020.
- [10] D. Fraunholz et al. Fingerprinting and evading honeypots. In *Proceedings of the International Conference on Security and Management*, 2017.
- [11] A. Name. Towards log analysis with ai agents: Cowrie case study. *Journal of Information Security*, 2025.
- [12] M. Hussain et al. Dionaea honeypot implementation and malware analysis in cloud environment. *Journal of Technical Computing*, 2024. URL <https://ejournal.unikl.edu.my/index.php/JTeC/article/view/674/677>.
- [13] J. Kelly et al. A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems. *IEEE Communications Surveys & Tutorials*, 23(4):2431–2458, 2021.
- [14] M. Antonakakis et al. Understanding the mirai botnet. In *26th USENIX Security Symposium*, pages 1093–1110, 2017.

- [15] M. Akbanov, V. G. Vassilakis, and M. D. Logothetis. Wannacry ransomware: Analysis of infection, persistence, recovery prevention and propagation mechanisms. *Journal of Telecommunications and Information Technology*, pages 113–121, 2019.
- [16] D. Braz. Honeypots in active defense: A tactical approach to cyber threat intelligence and deception. Msc thesis in telecommunications and informatics engineering, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal, 2024.
- [17] I. Toma et al. Intercepting and monitoring potentially malicious payloads with web honeypots. *Journal of Cybersecurity and Privacy*, 2025.
- [18] E. Vasilomanolakis et al. Towards bio-inspired cyber-deception: A case study of ssh and telnet honeypots. In *Proceedings of the International Conference on Cybersecurity*, 2025.
- [19] P. Bhandari et al. Q-cowrie: An adaptive honeypot to analyse attackers' behaviour. *Journal of Computer Virology and Hacking Techniques*, 2026. doi: 10.1007/s10207-026-01221-5.
- [20] F. Wibawa et al. Log analysis using elk stack. *IEEE Conference Publications*, 2021.
- [21] S. Bistarelli et al. A docker-based honeypot architecture. In *Proceedings of Cybersecurity Conference*, 2020.
- [22] VirusTotal. Virustotal api v3 documentation. <https://docs.virustotal.com/reference/overview>, 2026.